

Long Short Term Memory Network (LSTM)

Applications of RNN

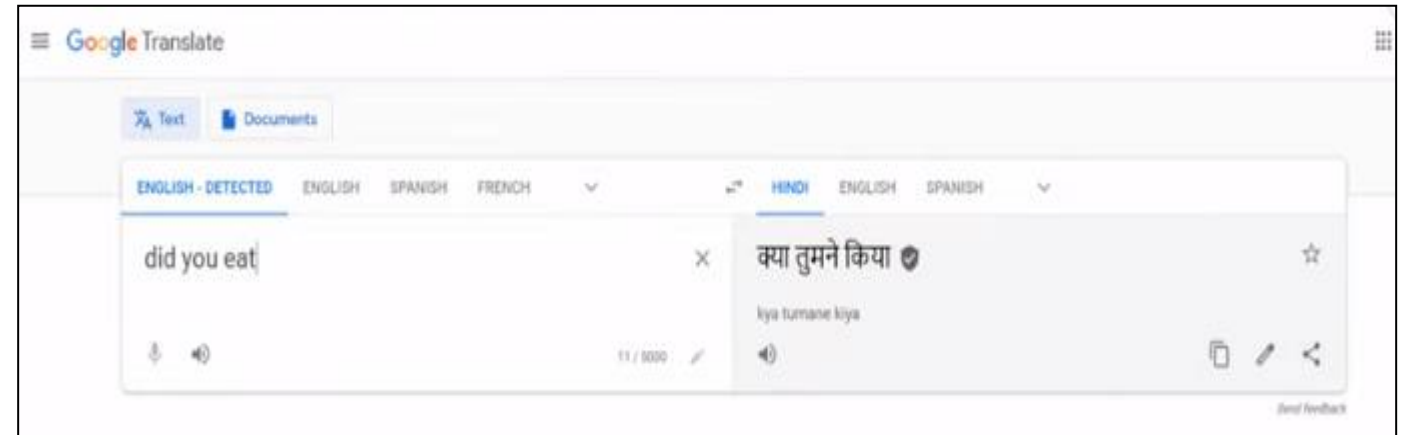
Speech Recognition



Language Translation



Prediction Problems



Speech Translation

	x		y
auto complete	not interested at	→	this time
translation	how are you?	→	क्या हाल है?
NER	Rudolph Smith bought 1000 shares of tesla Inc. in March 2020	→	Rudolph Smith bought 1000 shares of tesla Inc. in March 2020 Person Company time
Sentiment Analysis	Not only the fan was expensive, but it was broken when it arrived.	→	★☆☆☆☆

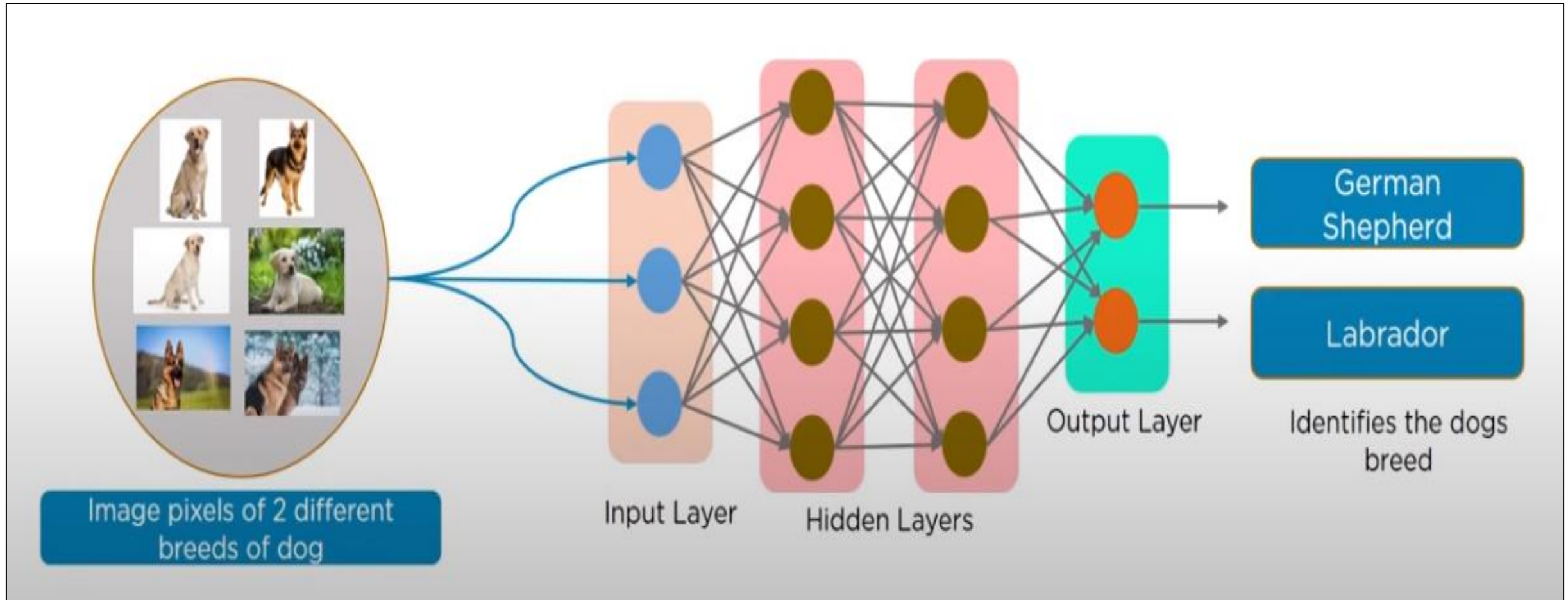


“A Dog catching a ball in mid air”

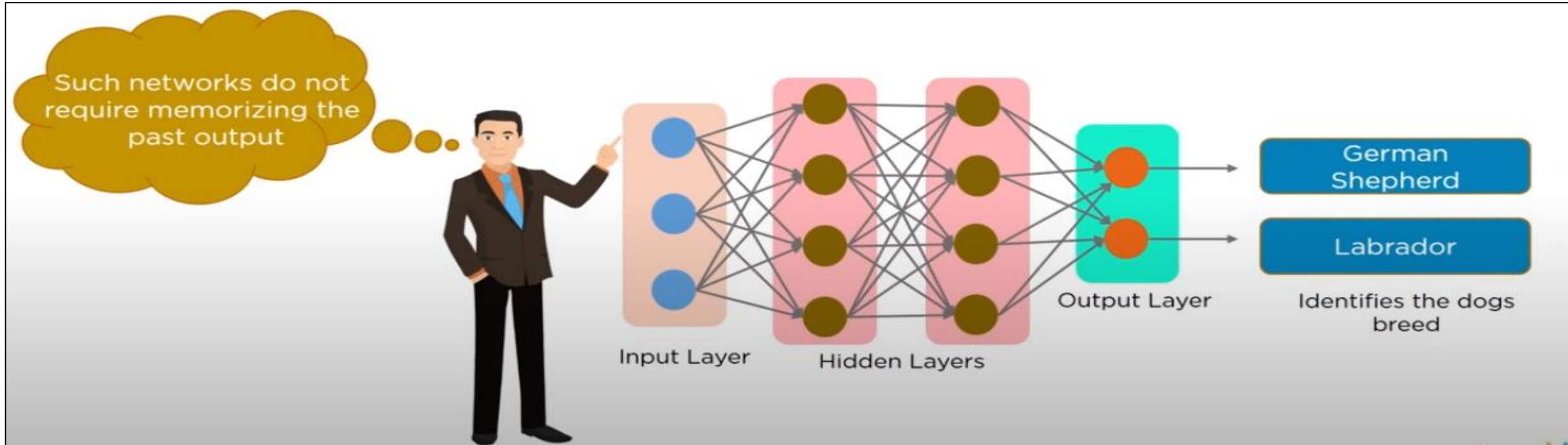
Image captioning

RNN is used to caption an image by analyzing the activities present in it

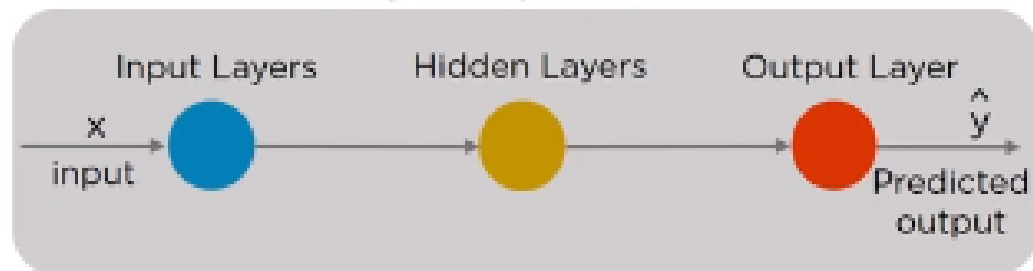
Recurrent Neural Networks(RNN)



Recurrent Neural Networks(RNN)



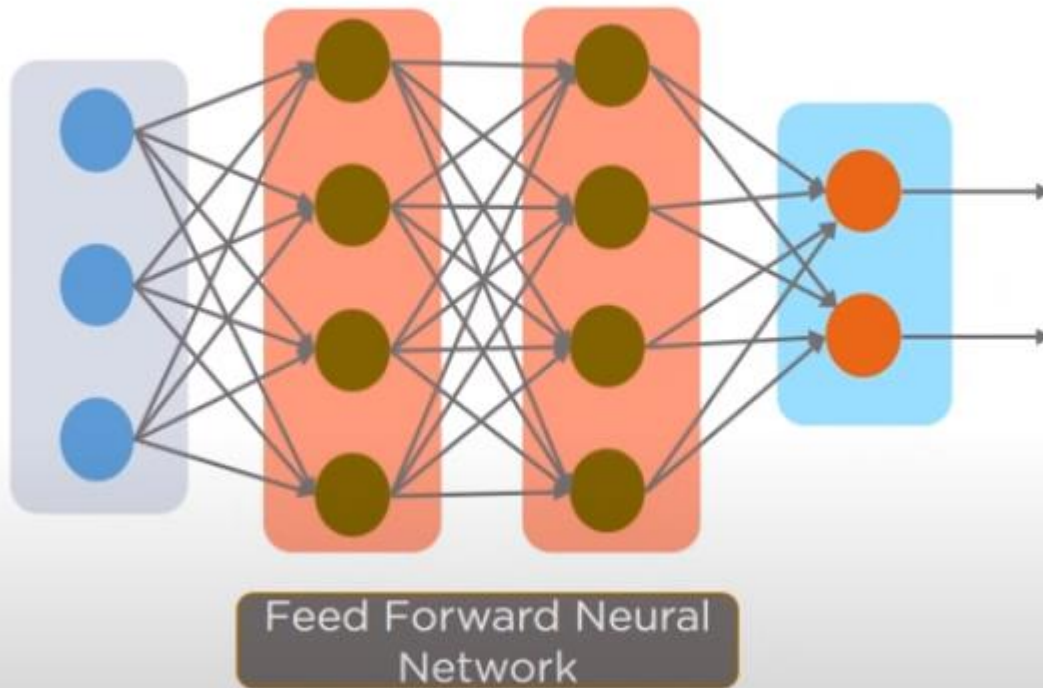
Simplified presentation



1. Decisions are based on the current input
2. No memory about the past
3. No Future scope

Why Recurrent Neural Networks(RNN)?

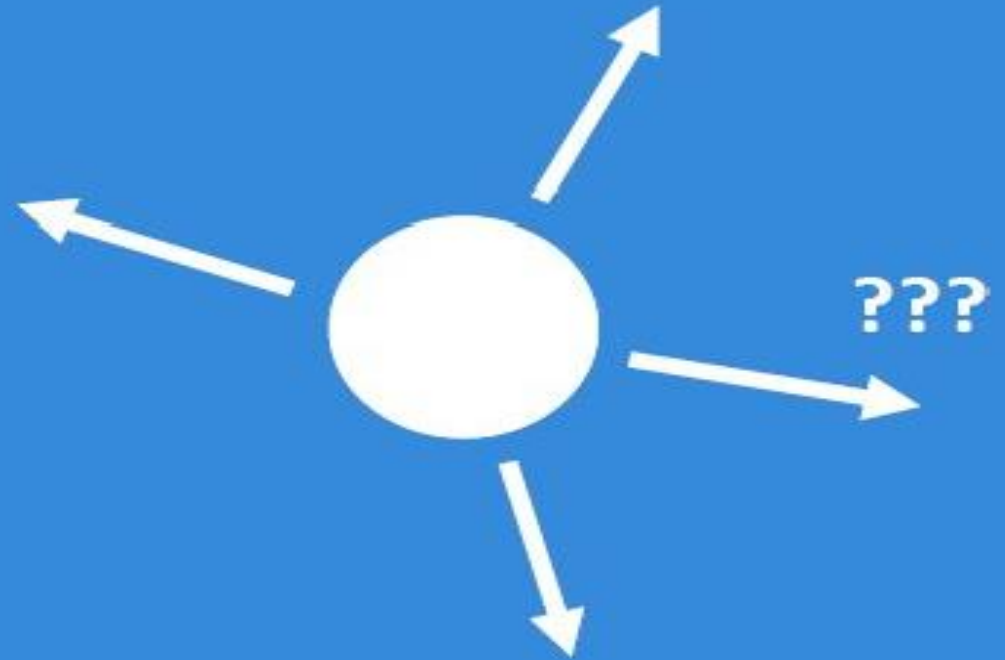
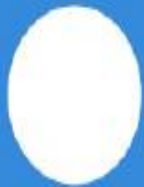
Issues in Feed Forward Neural Network



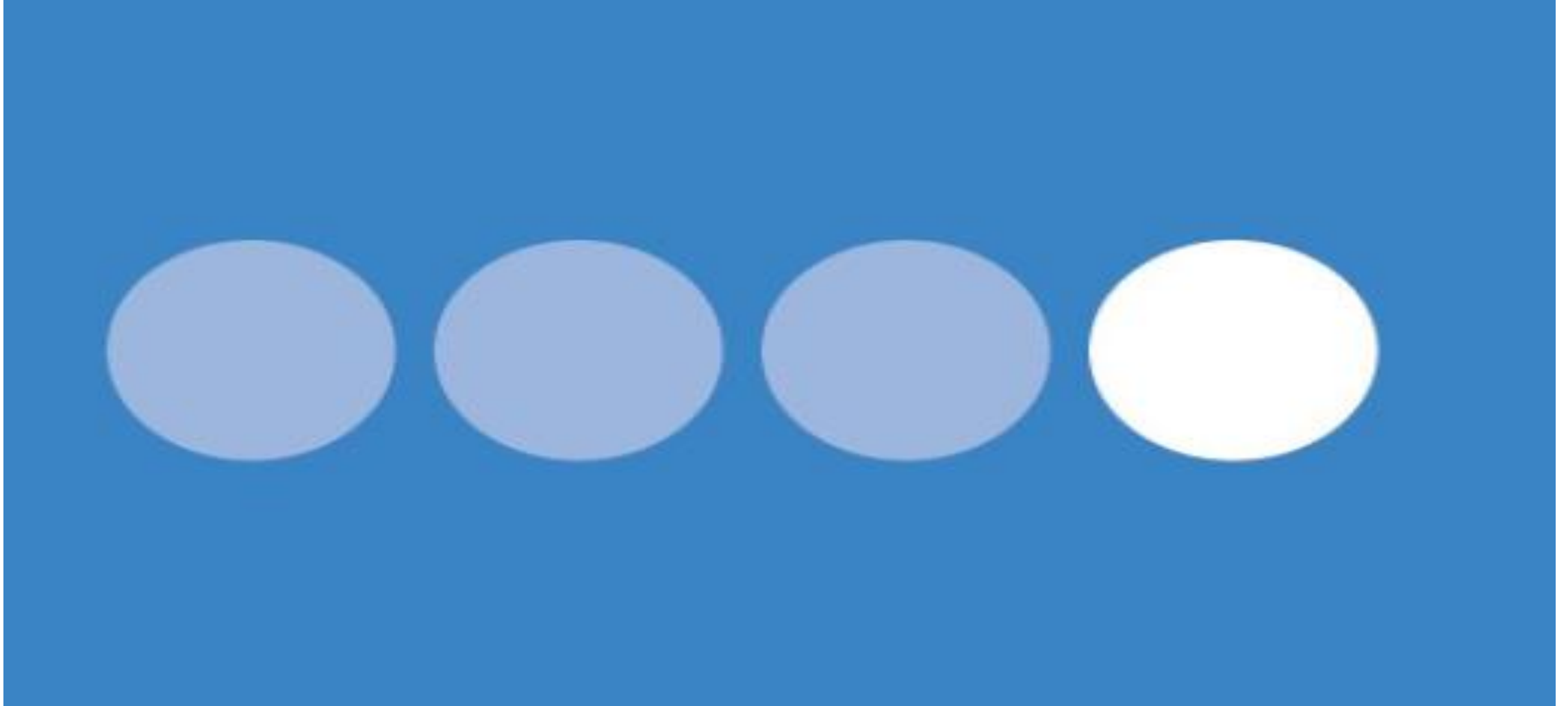
- 01 cannot handle sequential data
- 02 considers only the current input
- 03 cannot memorize previous inputs

Recurrent Neural Networks(RNN)

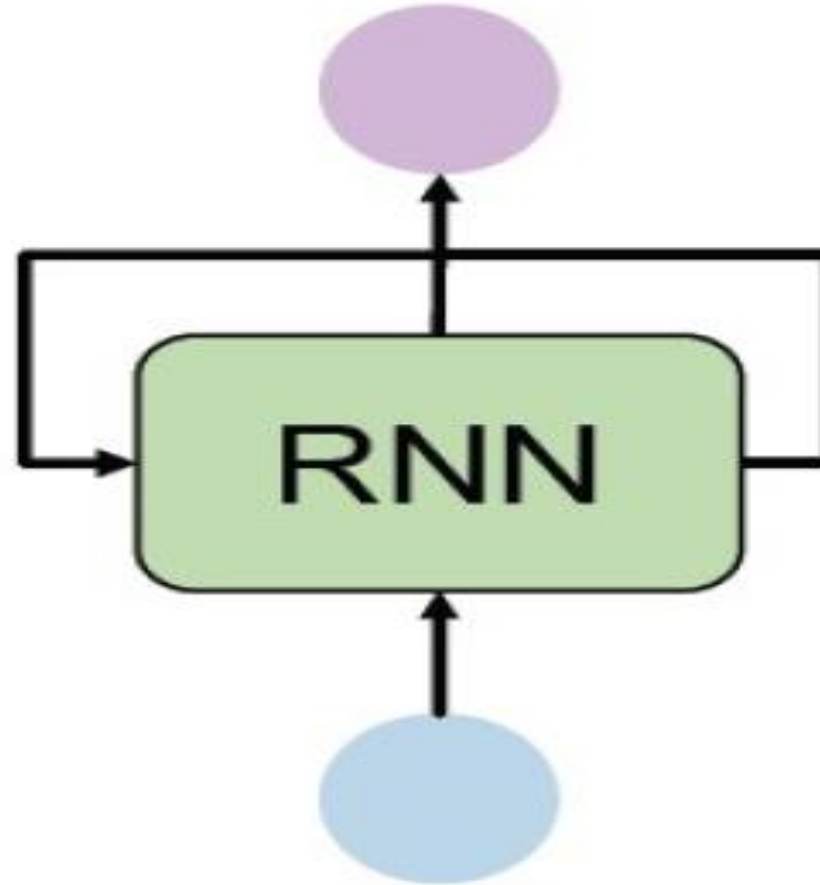
Given an image of a ball,
can you predict where it will go next?



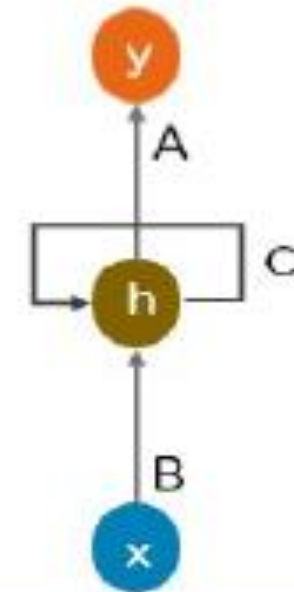
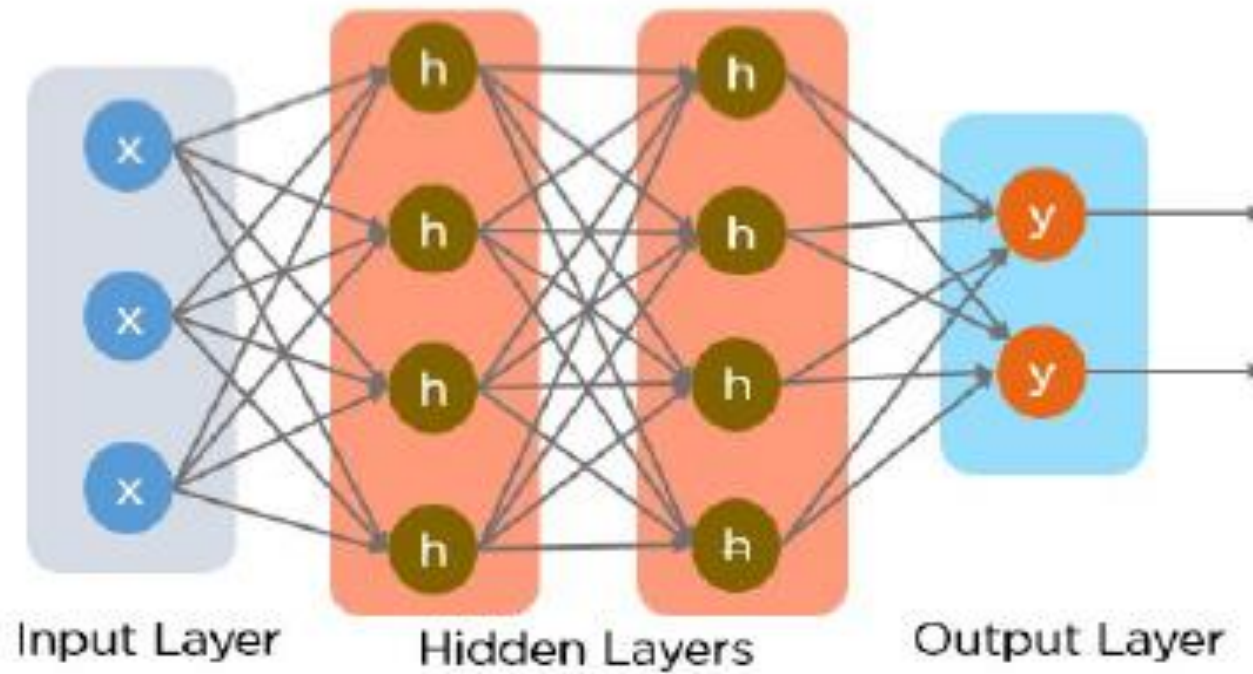
Recurrent Neural Networks(RNN)



Recurrent Neural Networks(RNN)

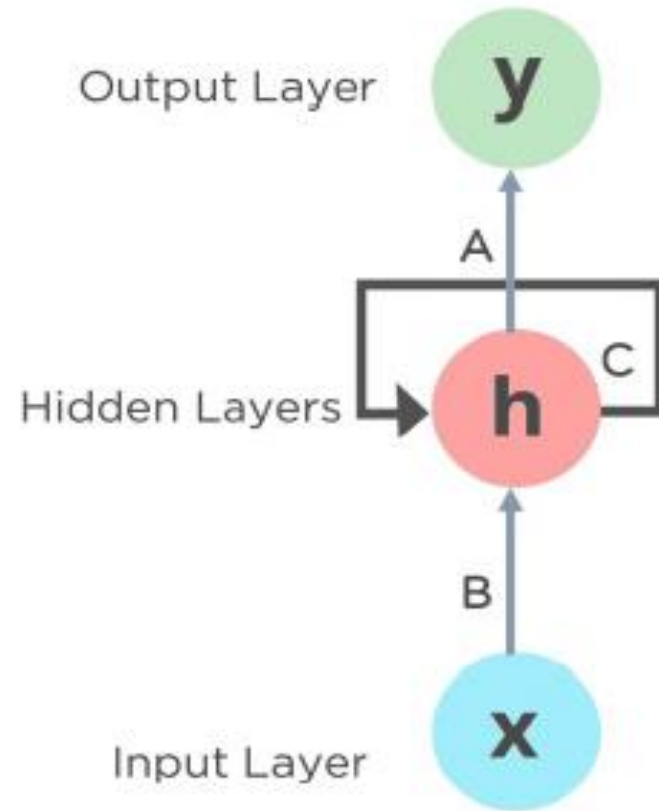


Recurrent Neural Networks(RNN)



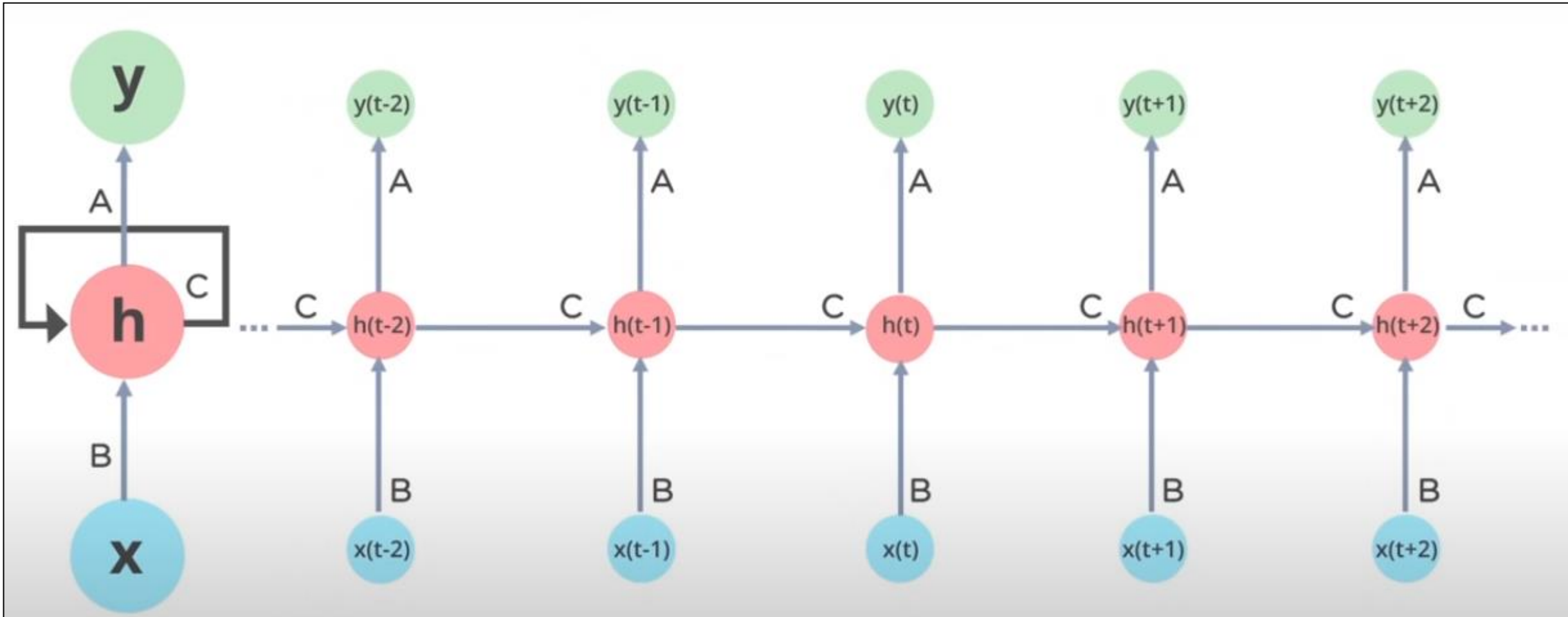
Recurrent Neural Network

Network Framework

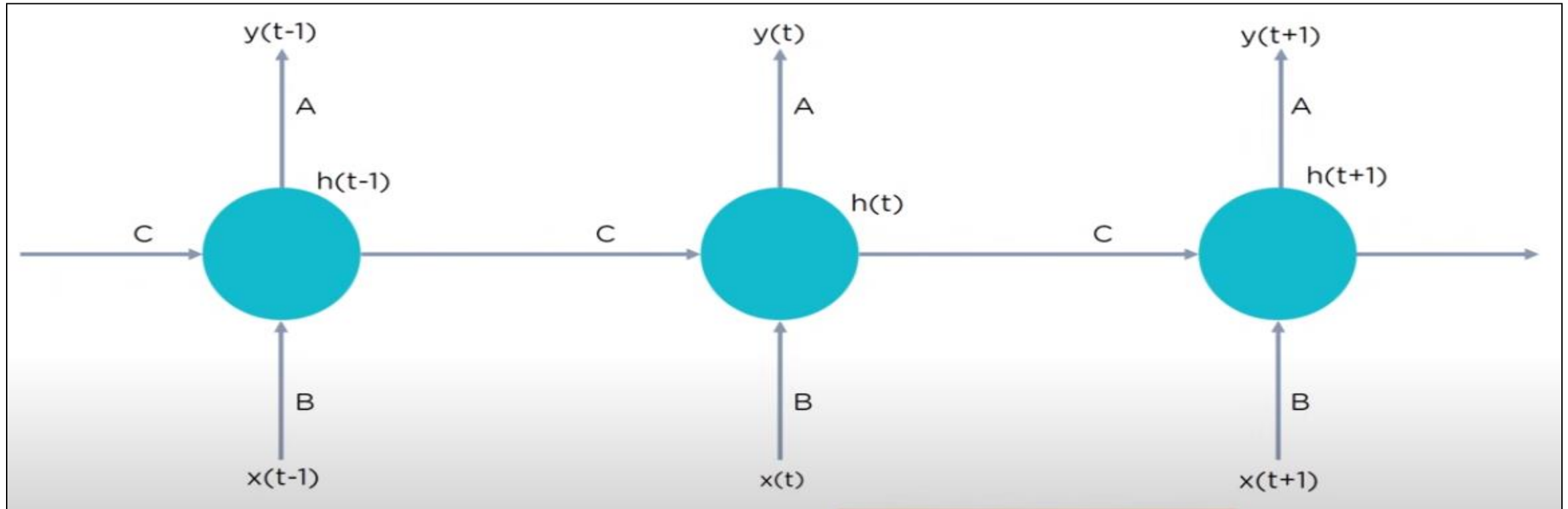


A, B and C are the parameters

Network Time Sequence Expansion



Recurrent Neural Networks(RNN)



$$h(t) = f_c(h(t-1), x(t))$$

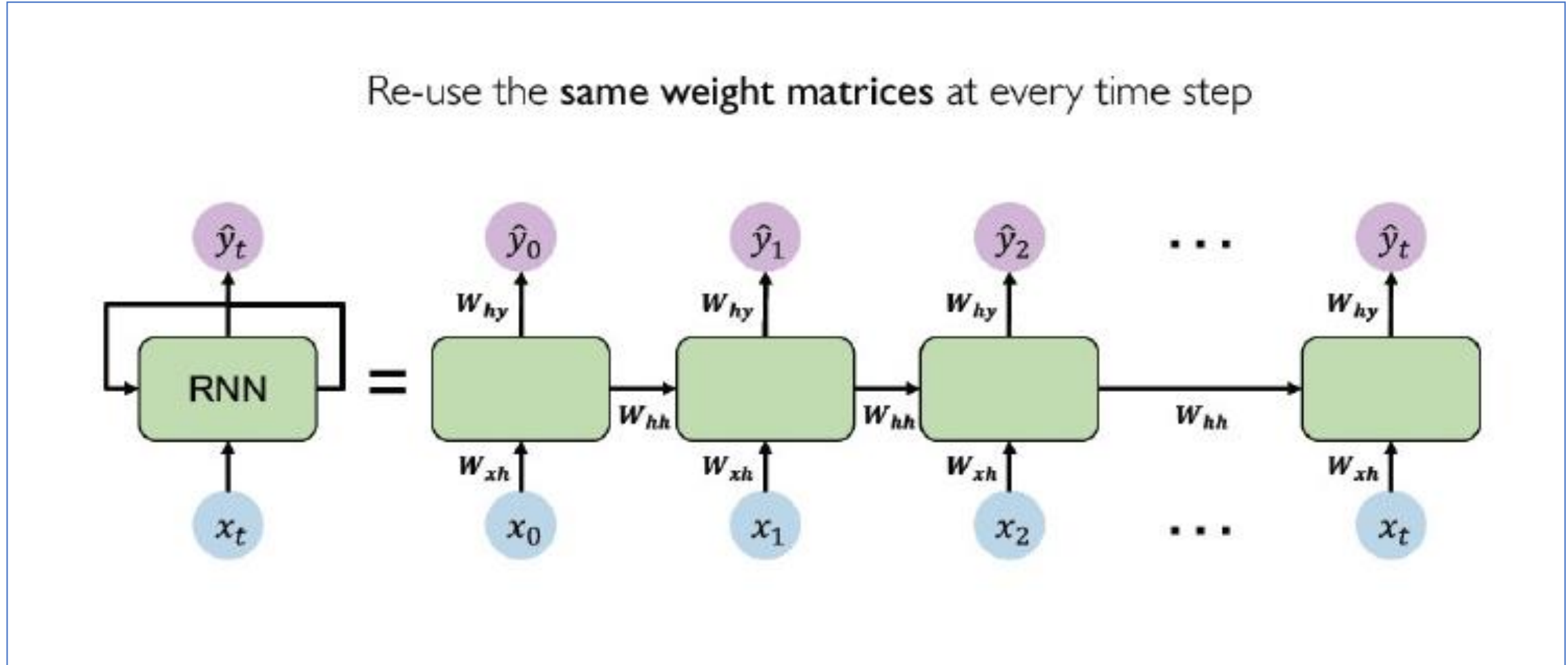
$h(t)$ = new state

f_c = function with parameter c

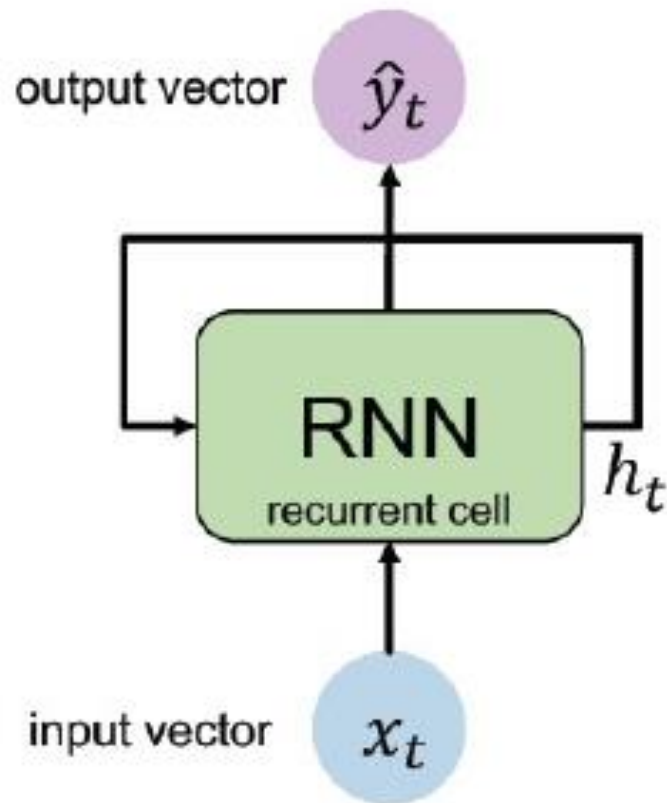
$h(t-1)$ = old state

$x(t)$ = input vector at time step t

RNN Weight Matrices



Recurrent Cell



Apply a **recurrence relation** at every time step to process a sequence:

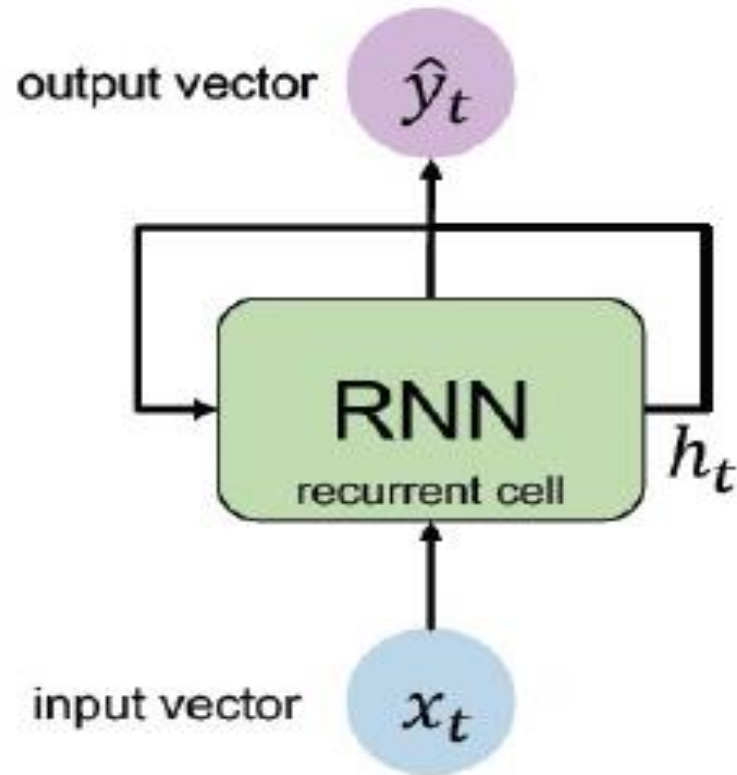
$$\boxed{h_t} = \boxed{f_W}(\boxed{h_{t-1}}, \boxed{x_t})$$

new state function parameterized by W old state input vector at time step t

Note: the same function and set of parameters are used at every time step

Recurrent Neural Networks(RNN)

Input, State Update, and Output



Output Vector

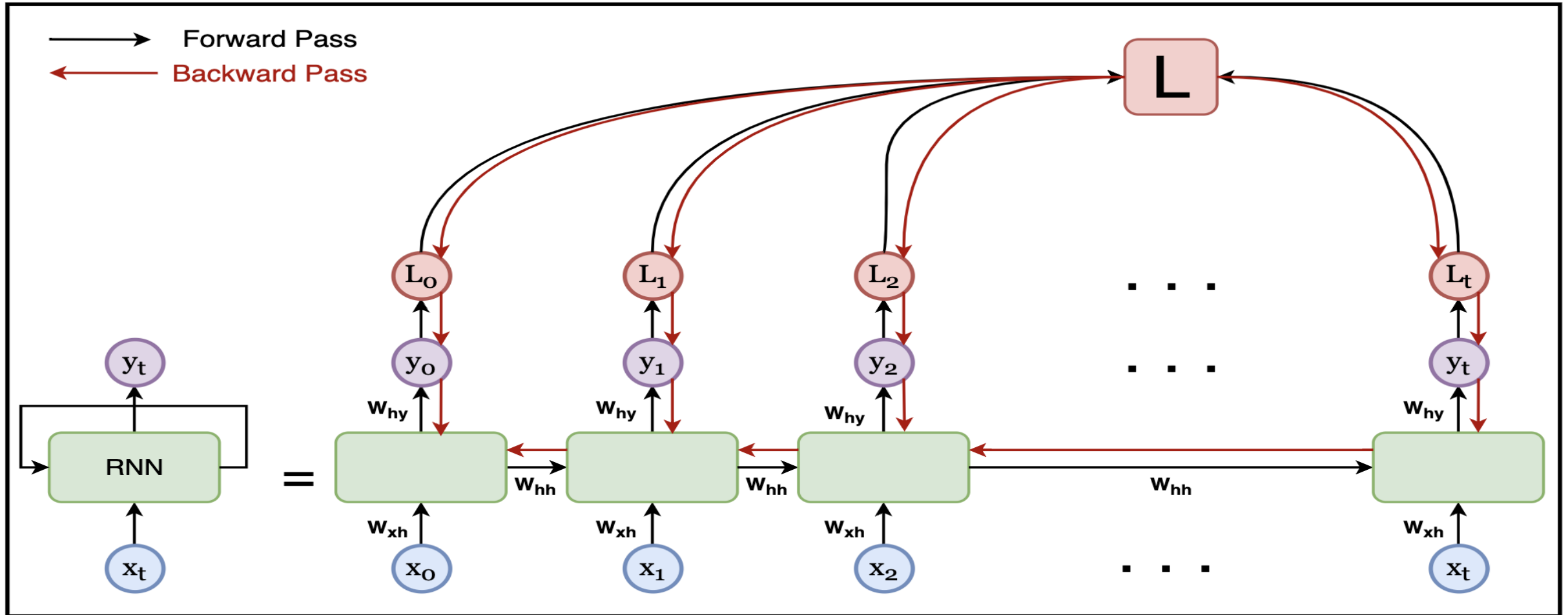
$$\hat{y}_t = W_{hy}h_t$$

Update Hidden State

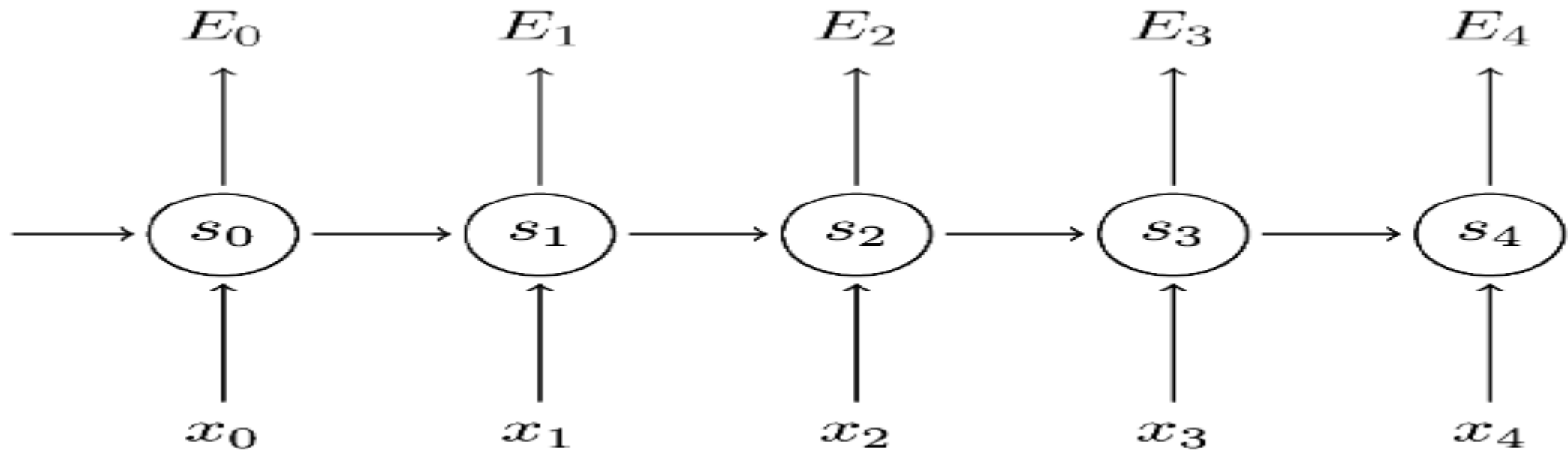
$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t)$$

Input Vector

Recurrent Neural Networks(RNN)



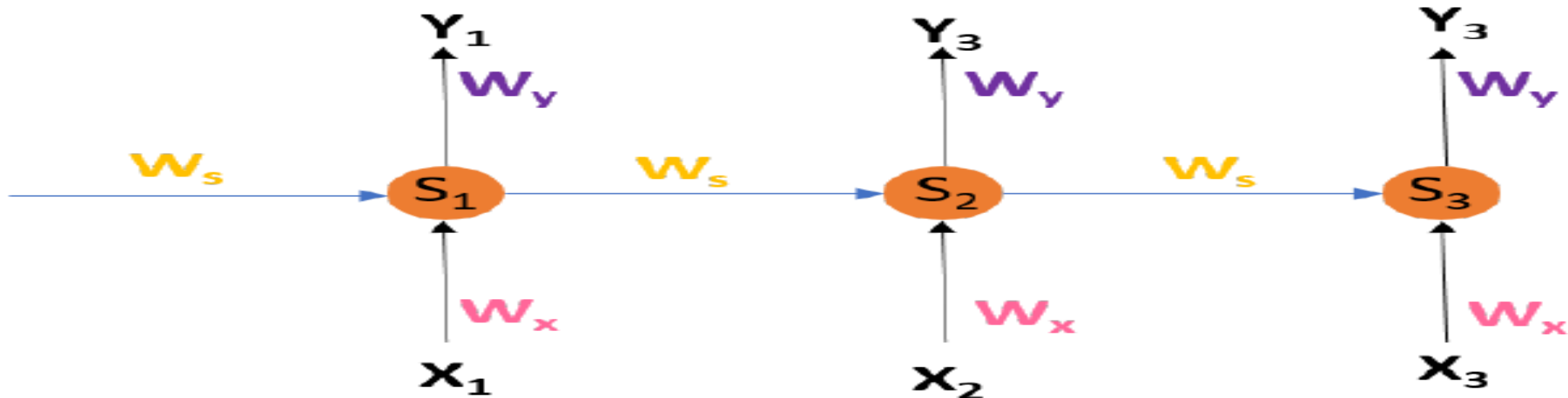
Backpropagation Through Time (BPTT)



$$\frac{\partial E}{\partial W} = \sum_t \frac{\partial E_t}{\partial W}$$

Recurrent Neural Networks(RNN)

BPTT



$$S_t = g_1(W_x x_t + W_s S_{t-1})$$

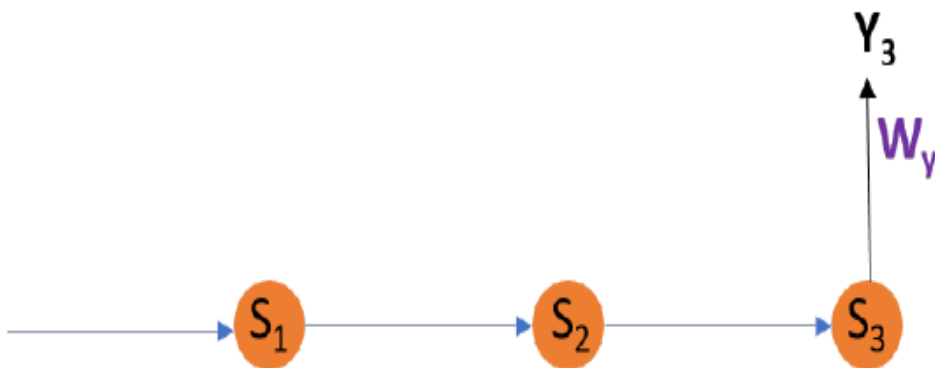
$$Y_t = g_2(W_Y S_t)$$

$$E_t = (d_t - Y_t)^2$$

$$E_3 = (d_3 - Y_3)^2$$

Adjusting W_y

- For better understanding, let us consider the following representation:



$$\frac{\partial E_3}{\partial W_y} = \frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial W_y}$$

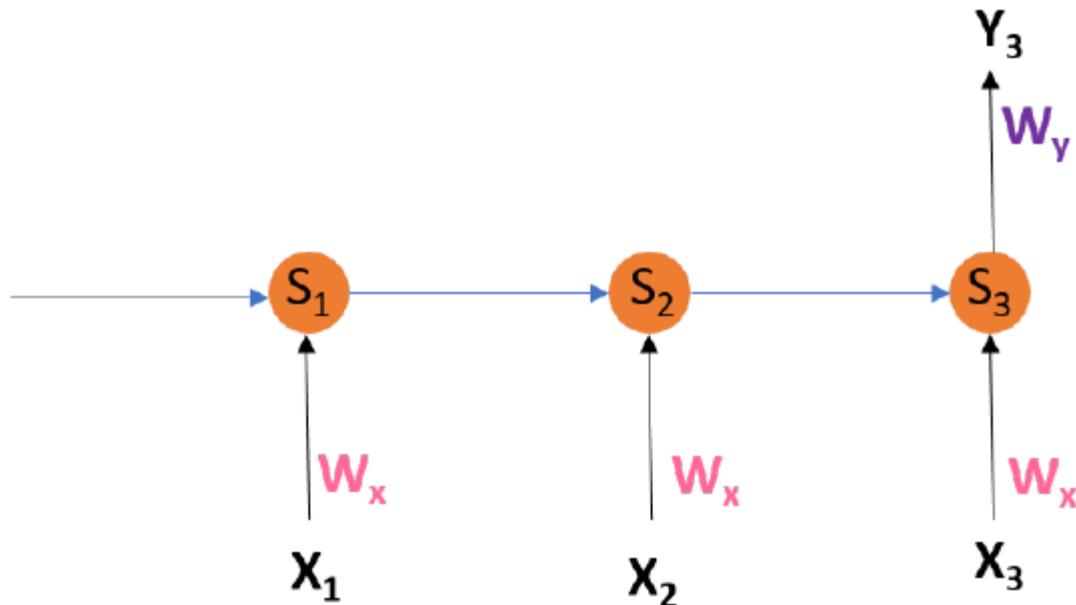
- Explanation:**
 E_3 is a function of Y_3 . Hence, we differentiate E_3 w.r.t. Y_3 .
 Y_3 is a function of W_y . Hence, we differentiate Y_3 w.r.t. W_y .

Adjusting W_s



$$\begin{aligned}\frac{\partial E_3}{\partial W_S} = & \left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_S} \right) + \\ & \left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_S} \right) + \\ & \left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_S} \right)\end{aligned}$$

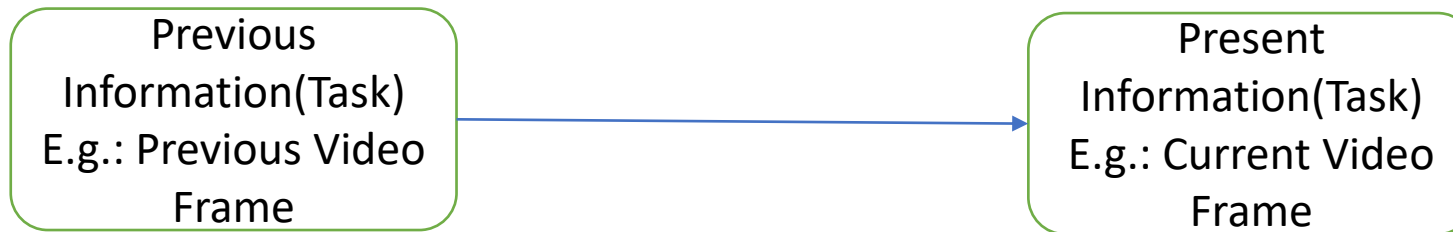
Adjusting W_X



Adjusting W_X

$$\frac{\partial E_3}{\partial W_X} = \left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial W_X} \right) + \left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial W_X} \right) + \left(\frac{\partial E_3}{\partial Y_3} \cdot \frac{\partial Y_3}{\partial S_3} \cdot \frac{\partial S_3}{\partial S_2} \cdot \frac{\partial S_2}{\partial S_1} \cdot \frac{\partial S_1}{\partial W_X} \right)$$

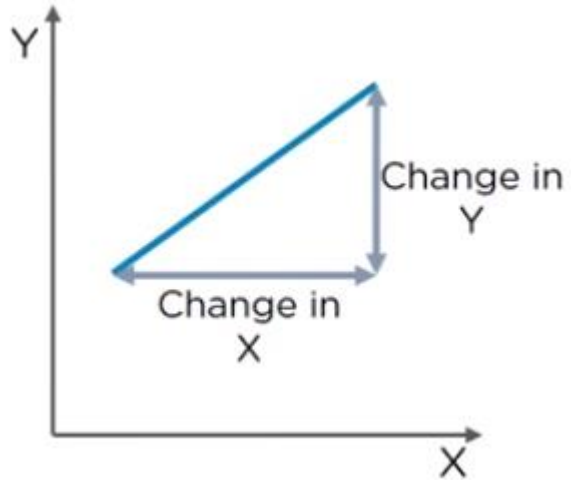
$$\frac{\partial E_N}{\partial W_S} = \sum_{i=1}^N \frac{\partial E_N}{\partial Y_N} \cdot \frac{\partial Y_N}{\partial S_i} \cdot \frac{\partial S_i}{\partial W_X}$$



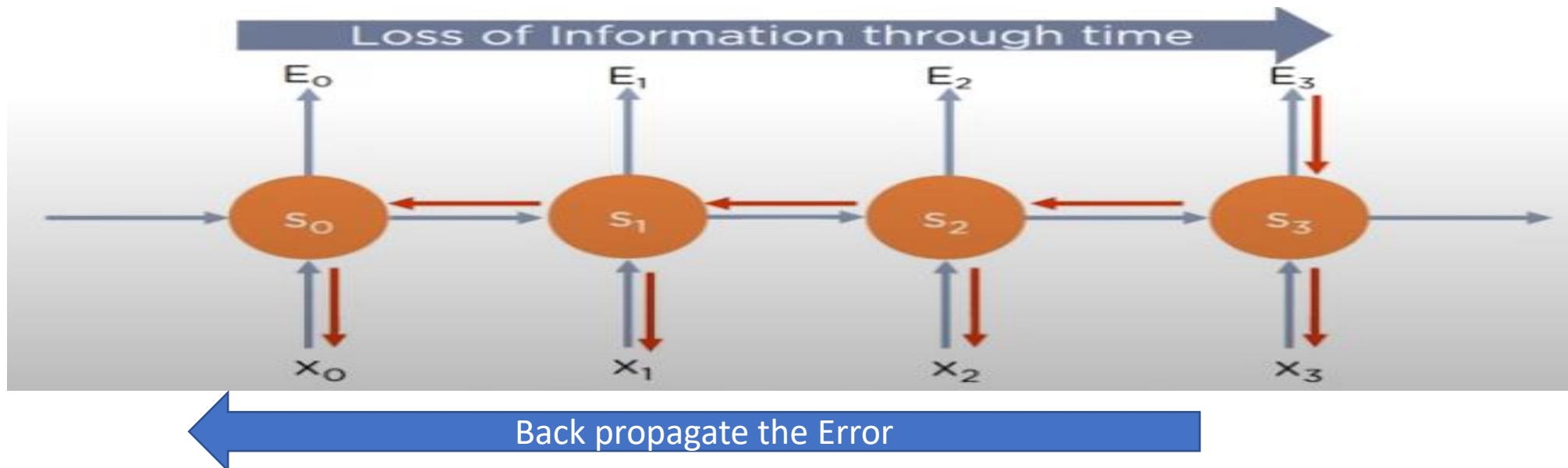
- Sometimes, we only need to look at recent information to perform the present task.
- For E.g.: “**The clouds are in the _____,**”
 - It’s pretty obvious the next word is going to be **sky**.
- In such cases, where the gap between the relevant information and the place that it’s needed is small, RNNs can learn to use the past information.

- Different Context:
 - Consider trying to predict the last word in the text “I grew up in Karnataka... I speak fluent Kannada.”
- Recent information suggests that the next word is probably the name of a language, but if we want to narrow down which language, we need the context of Karnataka, from further back.
- It’s entirely possible for the gap between the relevant information and the point where it is needed to become very large.
- **Unfortunately, as that gap grows, RNNs become unable to learn to connect the information.**

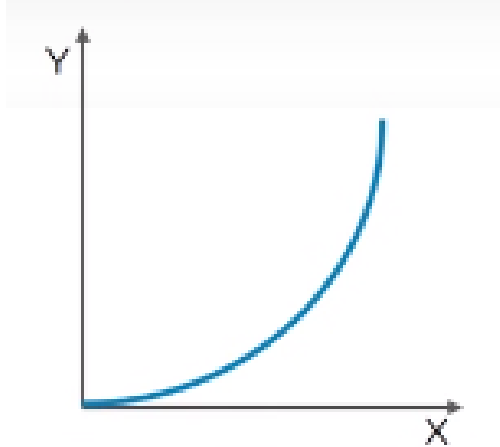
Vanishing Gradient Problem



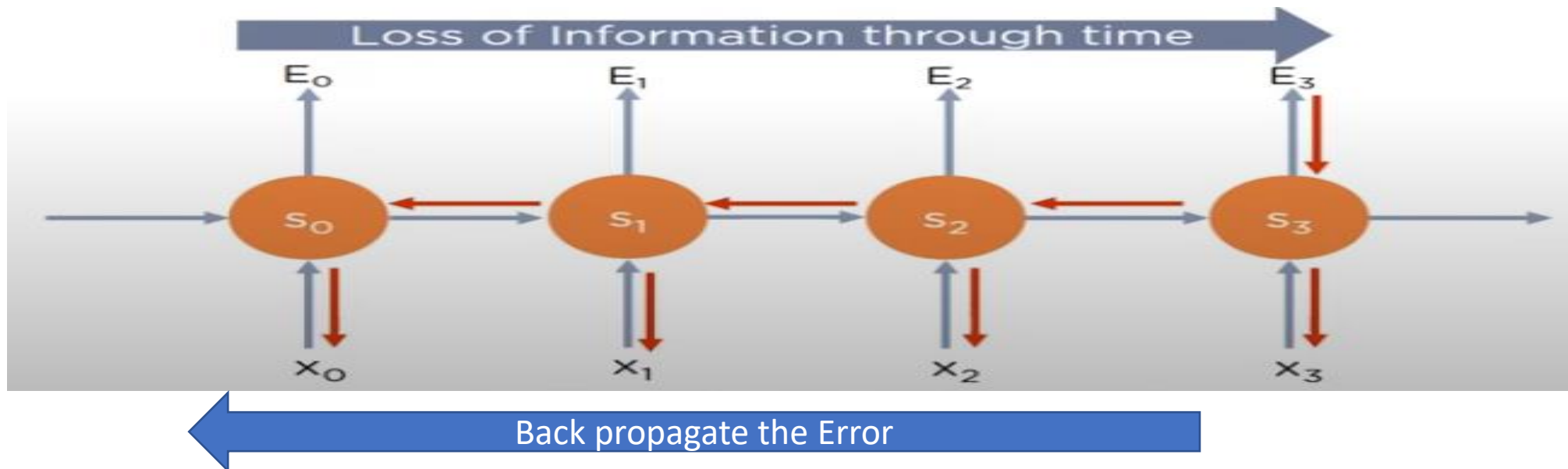
While training a RNN, your slope can be either too small or very large and this makes training difficult. When the slope is too small, the problem is known as *Vanishing gradient*.



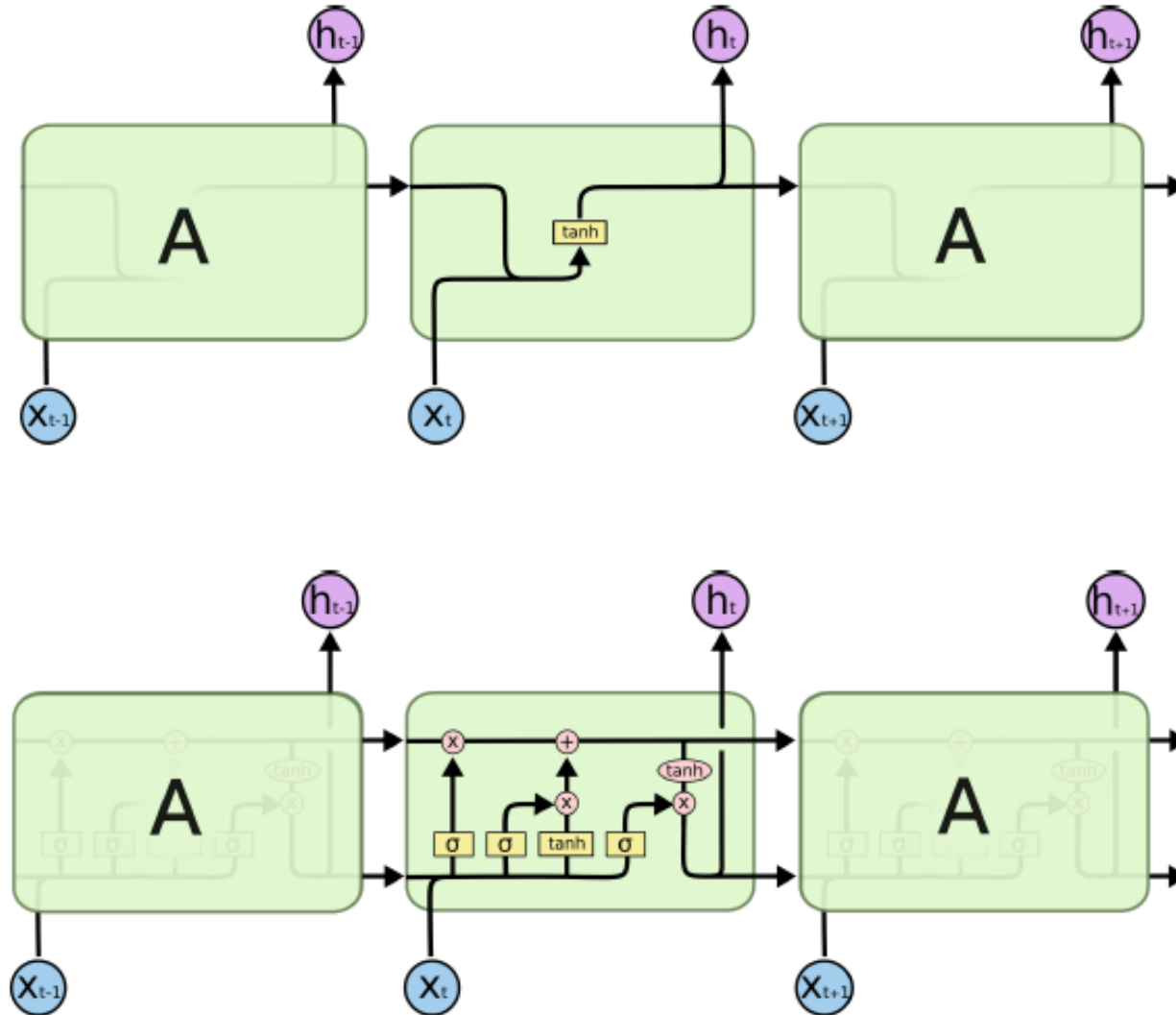
Exploding Gradient Problem



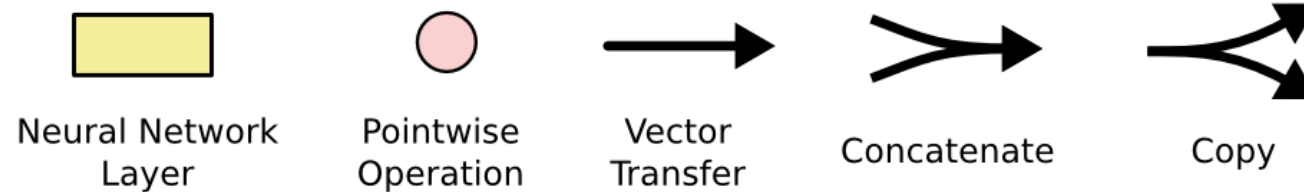
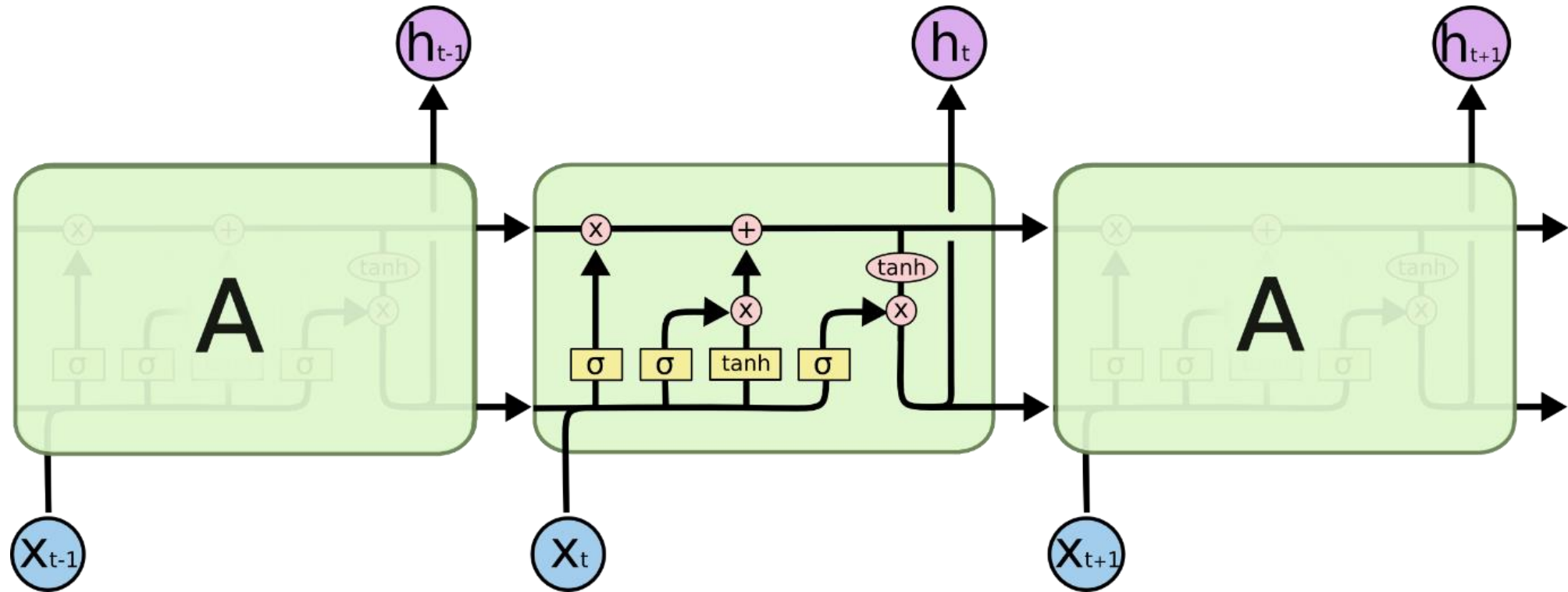
When the slope tends to grow exponentially instead of decaying, this problem is called *Exploding gradient*.



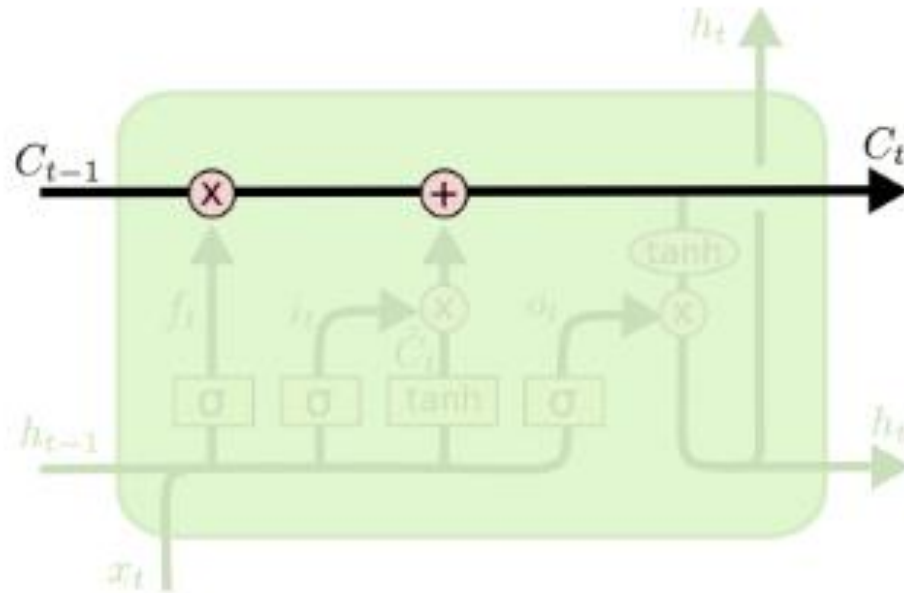
RNN vs LSTM



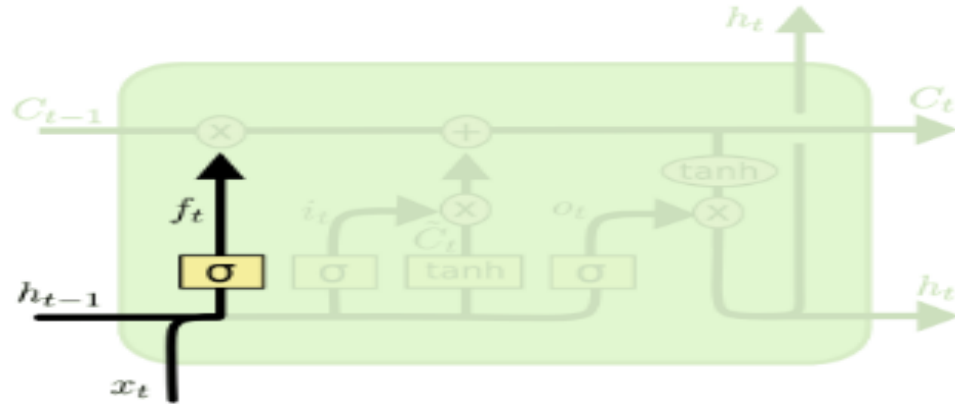
LSTM : Components



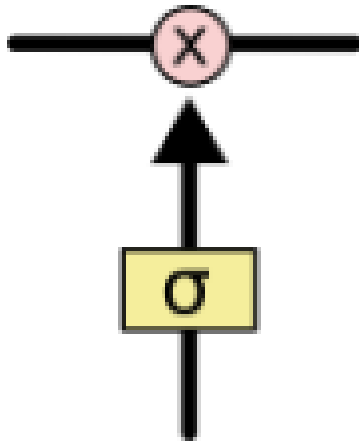
LSTM : Components – Memory Cell



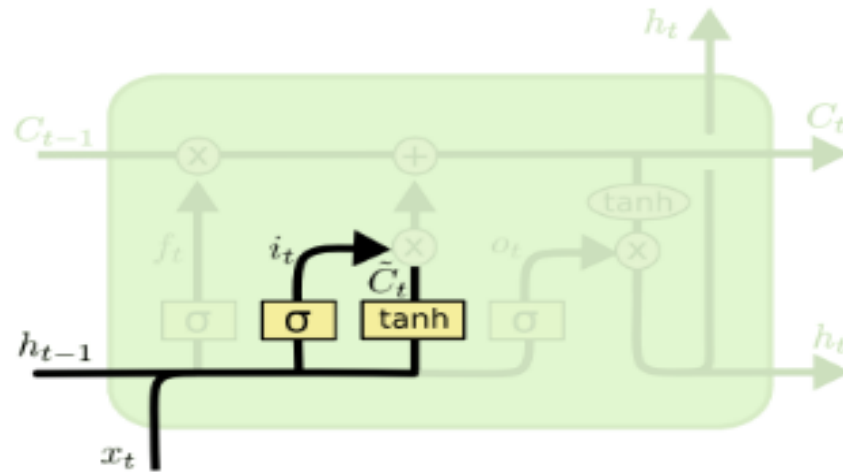
LSTM : Components – Forget Gate



$$f_t = \sigma (W_f \cdot [h_{t-1}, x_t] + b_f)$$

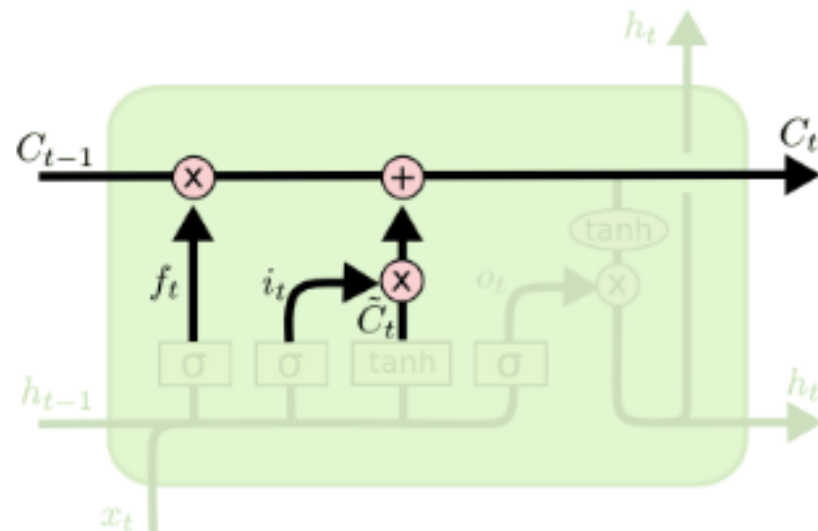


LSTM : Components – Input Gate



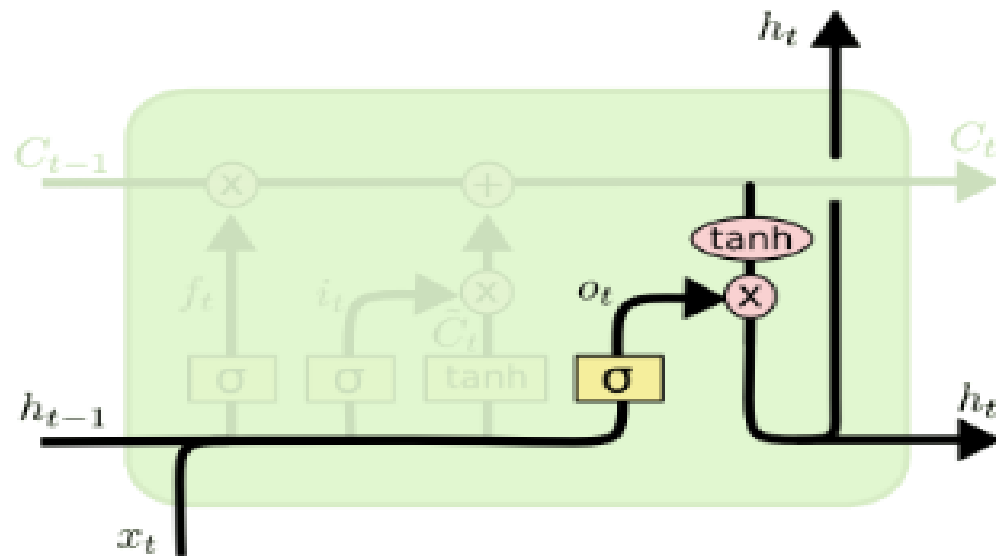
$$i_t = \sigma (W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM : Components – Output Gate



$$o_t = \sigma (W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh (C_t)$$